

B2B Webshop

Indledning

Formålet med dette projekt er at designe og implementere en effektiv databasearkitektur til en B2B-webshop applikation. Applikationen gør det muligt for brugere at tilgå og gennemse produkter uden at være logget ind, men kræver login for at kunne foretage køb. Derudover indeholder applikationen en administrationsgrænseflade, hvor en administrator kan udføre centrale funktioner som oprettelse, opdatering og sletning af brugere og produkter, samt få adgang til statistik over anvendelsen af systemet.

Fokus i udviklingen har været performance og skalerbarhed i databaselaget. Der er derfor arbejdet med teknikker som pagination og caching for at minimere unødvendige dataoperationer og forbedre svartider. Projektet anvender en polyglot persistence tilgang, hvor forskellige databaseteknologier er anvendt til forskellige dele af systemet.

Funktionelle krav:

- Som besøgende skal jeg kunne tilgå produkter uden login.
- Som registreret kunde skal jeg kunne logge ind og gennemføre ordrer.
- Som administrator skal jeg kunne oprette, redigere og deaktivere produkter og brugere.
- Som administrator skal jeg kunne se statistik over ordrer og brugere.

Ikke-funktionelle krav:

- Systemet skal understøtte høj ydeevne ved visning af produkter.
- Systemet skal kunne håndtere store mængder produktdata.
- Systemet skal udnytte caching og pagination til at reducere belastning af databaserne.

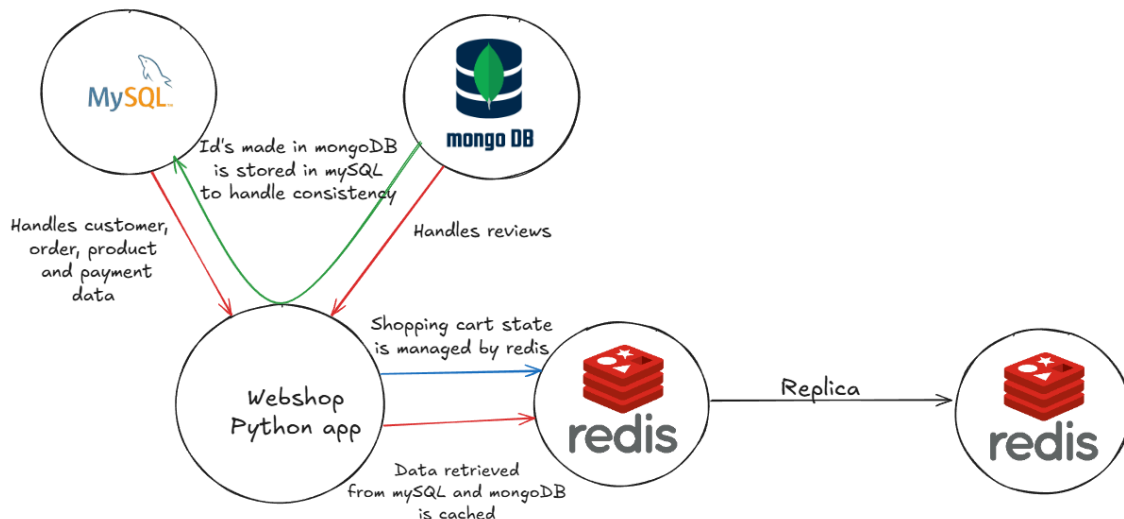
Første udkast til ER-diagram (se bilag 1.):

Det første konceptuelle design af systemet bestod af følgende entiteter:

- Customer, en registreret bruger, som kan foretage køb og efterlade reviews på platformen.
- Order, en ordre der indeholder produkter, pris, mm., men ikke nødvendigvis betalt.
- Product, det enkelte produkt, dækker over beskrivelsen af produktet og prisen.
- Review, en anmeldelse af et specifikt produkt, lavet af en specifik bruger.
- Payment, dokumentation for at en ordre er blevet betalt.

Arkitektur

Tilgangen til projektet blev at skabe en arkitektur, der anvendte flere databaseteknologier i samspil, for at opnå optimal performance. Kombinationen af styrkerne fra relationelle, dokumentbaserede og in-memory databaser har haft til hensigt at opfylde de funktionelle og ikke-funktionelle krav så effektivt som muligt.



Systemet gør brug af 3 databaser:

- MySQL (relationel) - der håndterer strukturerede og konsistenskrævende data som kunder, ordrer, produkter og betalinger.
- MongoDB (dokument) - anvendt til håndtering af produkt reviews, hvor skalerbarhed og ustruktureret tekst er i fokus.
- Redis (in-memory) - anvendt til både caching og state håndtering af indkøbskurven.

Python-applikationen fungerer som bindeled mellem databaserne og styrer læse-/skriveadgang, cache-opdateringer og konsistens mellem systemerne ved f.eks. at sikre, at relevante ID'er (som oprettes i MongoDB) også synkroniseres i MySQL.

Database design og implementering

Ved design af datamodellen har fokus været på performance, skalerbarhed og at udnytte databasernes individuelle styrker, med henblik på at håndtere struktureret, ustruktureret og sessionsbunden data effektivt.

MySQL

Udgangspunktet for datamodellen er, at tabellerne er normaliseret efter 3. normalform og er designet med transaktions sikkerhed i fokus. I et enkelt tilfælde blev normalformen dog brudt, da det for at undgå at lave unødigt mange joins, blev besluttet at denormalisere orders tabellen, ved at indsætte attributen `total_price`, som er direkte afhængig af data der findes i `order_lines` tabellen. I dette tilfælde benyttes triggers til at sørge for at der altid er konsistens mellem `total_price` attributen og summen af ordrelinjerne tilknyttet en ordre.

En særlig designbeslutning er brugen af stored procedures til håndtering af forretningslogik. De benyttes eksempelvis ved oprettelse af nye ordrer, hvor der indsættes data i flere forskellige tabeller på en gang og derfor er behov for at sikre at operationerne afvikles som én transaktion, med eventuel rollback, hvis noget går galt. **Se Bilag 2**

For at håndtere den store mængde af produkter blev der implementeret en stored procedure, som kombinerer søgning med cursor baseret pagination, der i kombination med caching via Redis, skal reducere antallet af dyre databasekald, samt forbedre responsiviteten. **Se Bilag 3**

Der blev oprettet en række views med henblik på at gøre reducere kompleksiteten af de queries som sendes fra applikationslaget, hvilket giver effektiv adgang til hyppigt efterspurgte data, ved hjælp af de eksisterende indekser, som f.eks. når en kunde efterspørger sine forhenværende ordrer. **Se Bilag 4**

Soft delete pattern er implementeret for at bevare relationer mellem tabellerne, sådan at data stadig er validt, selv hvis produkter udgår eller brugere bliver slettet, i forhold til salgshistorik eller andet. Derfor har en del af entiteterne fået en attribute is_active, som bruges til håndtering af soft deletion.

For fuld overblik over MySQL implementation **se bilag 5**, for det endelige ER-diagram og https://github.com/MrJustMeDahl/DB_webshop/blob/main/mysql_creation_script.sql for det fulde SQL creation script.

Redis

Redis blev valgt til to centrale funktioner: kundernes indkøbskurve og caching af søgeresultater. Her sørges altså for at midlertidige data, som indkøbskurven, hurtigt kan opdateres, mens at de automatisk fjernes igen, hvis de bliver glemt når dataens time to live løber ud.

Caching af søgeresultater bidrager til at undgå unødige kald til MySQL databasen, da det er hurtigere at finde dataen in-memory. Grundet tilgangen med at benytte pagination er det altså hele pages der bliver cachet, og tanken bag dette er at det vil komme alle brugerne af systemet til gode, da de mest populære page opslag vil ende med oftest at være cachet og derved kan leveres meget hurtigt til klienten.

```
def cache_products(cache_key, products, has_more, ttl=300)
    redis_conn = get_redis_connection()
    if redis_conn:
        redis_conn.setex(cache_key, ttl, json.dumps({
            "products": products,
            "has_more": has_more
        })))
```

Udfordringen ved at bruge Redis til at cache alle søgeresultaterne er dog at når entiteter opdateres, så skal cachen også opdateres, så der ikke kommer en naturlig forsinkelse mellem at en opdatering sættes i kraft og at opdateringen vises til klienterne. I denne case

hvor det er hele pages der bliver cachet er der dog ikke mulighed for at finde specifikke entries og derfor slettes hele cachen for den opdaterede entitet.

For at sikre stabilitet for disse 2 kernefunktionaliteter er Redis implementeret med en master og replica node. Tanken bag dette er at have en replica der skal tage over hvis master noden går ned, så der ikke pludseligt opstår performance problemer, hvis hele cachen forsvinder på en gang, for ikke at tale om den dårlige brugeroplevelse forbundet med at ens indkøbskurv pludseligt forsvinder, hvorefter det vil være umuligt at handle.

Mongodb

MongoDB blev brugt til at holde webshoppens "reviews" for produkterne. Teksterne gemmes som dokumenter i MongoDB, mens relaterede metadata som rating, customer_id og product_id refereres via Mysql databasen. Dette gør det muligt at opretholde den relationelle integritet fra MySQL og samtidig udnytte MongoDBs styrke i at håndtere ustrukturerede tekster.

Denne adskillelse af data betyder at ved visning af reviews er det kun relevante ID's der bliver hentet ud af MySQL, hvorefter disse ID's målrettet kan bruges til at hente review teksten i MongoDB, hvilket bevirker til at nedbringe mængden af data der overføres. Udover det vil opdelingen give mulighed for lettere at skalere databasen horisontalt, da MongoDB's sharding funktionalitet gør det let at sprede data ud over flere nodes, hvis det skulle blive nødvendigt i fremtiden. Det blev anset for at være en god forholdsregel, fordi review data potentielt er uendeligt i størrelse.

MongoDB's aggregation pipelines blev benyttet til statistiske udtræk, eksempelvis hvor lange reviews er i gennemsnit. **Se bilag 6**

Evaluering

Ved valget af databaser har der været stort fokus på performance og opskalering, hvilket afspejler sig i implementeringen. Spørgsmålet er dog, om performance er blevet vægtet for højt, taget i betragtning af at produktet er en B2B webshop. Man skal derfor overveje, om man skulle have haft mere fokus på konsistens og mindre på performance. Man kan på samme tid få foretaget hastighedstest, for at konkludere, om de inkluderede performance optimeringer der er foretaget har en stor nok effekt, eller om man skulle simplificere.

Undervejs i udviklingen blev vi opmærksomme på en designmæssig udfordring, som opstod som følge af vores fokus på performance. For at opnå hurtigere svartider havde vi valgt at gemme produktdata midlertidigt i både Redis (til caching) og i brugerens session storage. Denne tilgang skabte dog et problem i forhold til konsistens ved opdatering, oprettelse eller sletning af produkter.

Da session storage ligger hos brugeren, har vi ingen mulighed for at opdatere eller invalidere data, som allerede er hentet og lagret. Det betyder, at brugere potentielt kan have adgang til forældede produktdata. Hvis disse data tilføjes til kurven, bliver de igen cachet og potentielt skrevet tilbage i databasen, hvilket skaber risiko for ikke-konsistent data.

En løsning kunne være at rydde al cache, men dette ville samtidig slette alle aktive kurve for alle brugere, hvilket ikke er en god løsning for en webshop. Vi erkendte derfor, at produktdata ikke bør gemmes i session storage, og at sessionen udelukkende bør anvendes til UI-relateret tilstandsstyring. I en fremtidig version af systemet ville vi ændre designet, så produktdata altid hentes fra Redis eller direkte fra databasen, hvilket sikrer bedre datakonsistens og skalerbarhed.

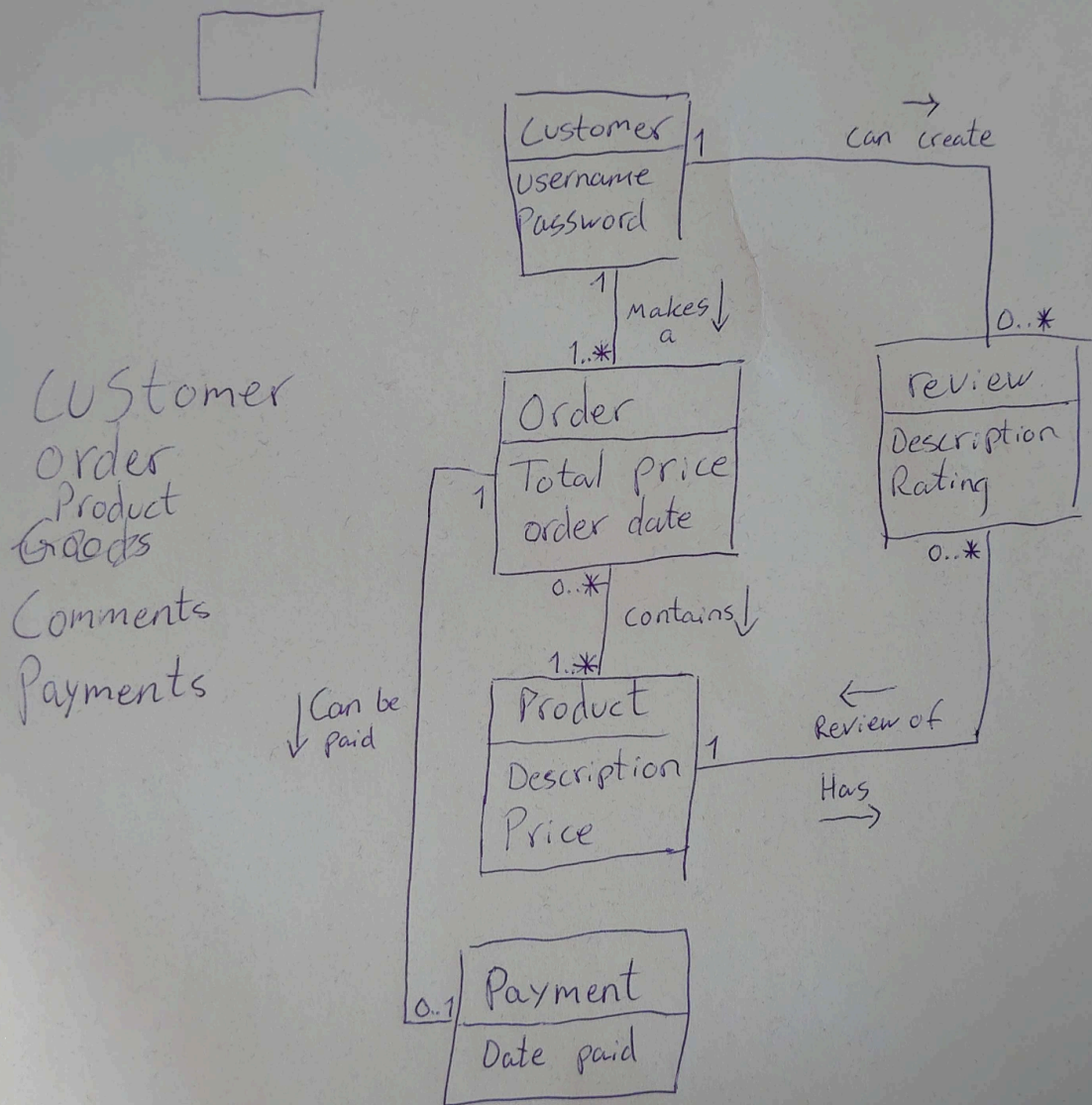
Ses der bort fra at beslutningen om at bruge session storage til opbevaring af produktdata, overraskede brugen af Redis til caching og sessionsstyring af indkøbskurvene dog positivt, da det virkede ekstremt hurtigt, og let at sætte op.

I forbindelse med valget om at lægge forretningslogik ned i databaselaget, gav det god mening med henblik på målet om at optimere performance, da vi undgår en række netværksskald og dataoverførsler mellem database og applikation. Det betød dog en mindre øget kompleksitet, da der nu skulle tages hensyn til - eksempelvis ikke at oprette triggers, der kan udløse hinanden og derved føre til utilsigtede kædereaktioner. Det anses som udgangspunkt ikke for et problem i denne case at der er meget tæt tilknytning mellem databasen og applikationen, da der er lagt megen tanke i at hver database bruges til dens styrke og derfor ikke sandsynligt bør udskiftes.

Løsningen lever op til de opstillede funktionelle og ikke-funktionelle krav, og demonstrerer hvordan en bevidst anvendelse af polyglot persistence kan understøtte en kompleks applikation med realistisk funktionalitet og teknisk dybde.

Projektet fremhæver det stærke samspil mellem de forskellige databasetyper, hvor hver teknologi udnyttes der, hvor den var mest effektiv. Det viste sig overraskende nemt at få disse systemer til at fungere sammen i praksis, og integrationen mellem dem skabte et stabilt setup, der kørte uden store problemer.

Bilag
Bilag 1.



Bilag 2

```
DELIMITER $$
USE `webshop_db`$$
CREATE PROCEDURE create_order_with_line(
    IN p_customer_id INT,
    IN p_product_id VARCHAR(20),
    IN p_quantity INT,
    IN p_price FLOAT,
    INOUT p_order_id INT
)
BEGIN
    DECLARE new_order_id INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;

    START TRANSACTION;

    IF p_order_id = 0 THEN
        SELECT IFNULL(MAX(order_id), 0) + 1 INTO new_order_id FROM orders;

        INSERT INTO orders (order_id, customer_id)
        VALUES (new_order_id, p_customer_id);

        SET p_order_id = new_order_id;
    END IF;

    INSERT INTO order_lines (order_id, product_id, quantity, price)
    VALUES (p_order_id, p_product_id, p_quantity, p_price);

    COMMIT;
END$$
```

Bilag 3

```
DELIMITER $$
USE `webshop_db`$$
CREATE PROCEDURE pagination(
    IN size INT,
    IN anchor_id varchar(45),
    IN search_filter varchar(45),
    IN direction ENUM('next', 'prev')
)
BEGIN
    IF direction = 'next' THEN
        SELECT * FROM products as p
        WHERE p.description LIKE CONCAT('%', search_filter, '%')
        AND (anchor_id IS NULL OR p.product_id > anchor_id)
        ORDER BY p.product_id ASC
        LIMIT size;
    ELSE
        SELECT * FROM products as p
        WHERE p.description LIKE CONCAT('%', search_filter, '%')
        AND (anchor_id IS NULL OR p.product_id < anchor_id)
        ORDER BY p.product_id ASC
        LIMIT size;
    END IF;
END$$

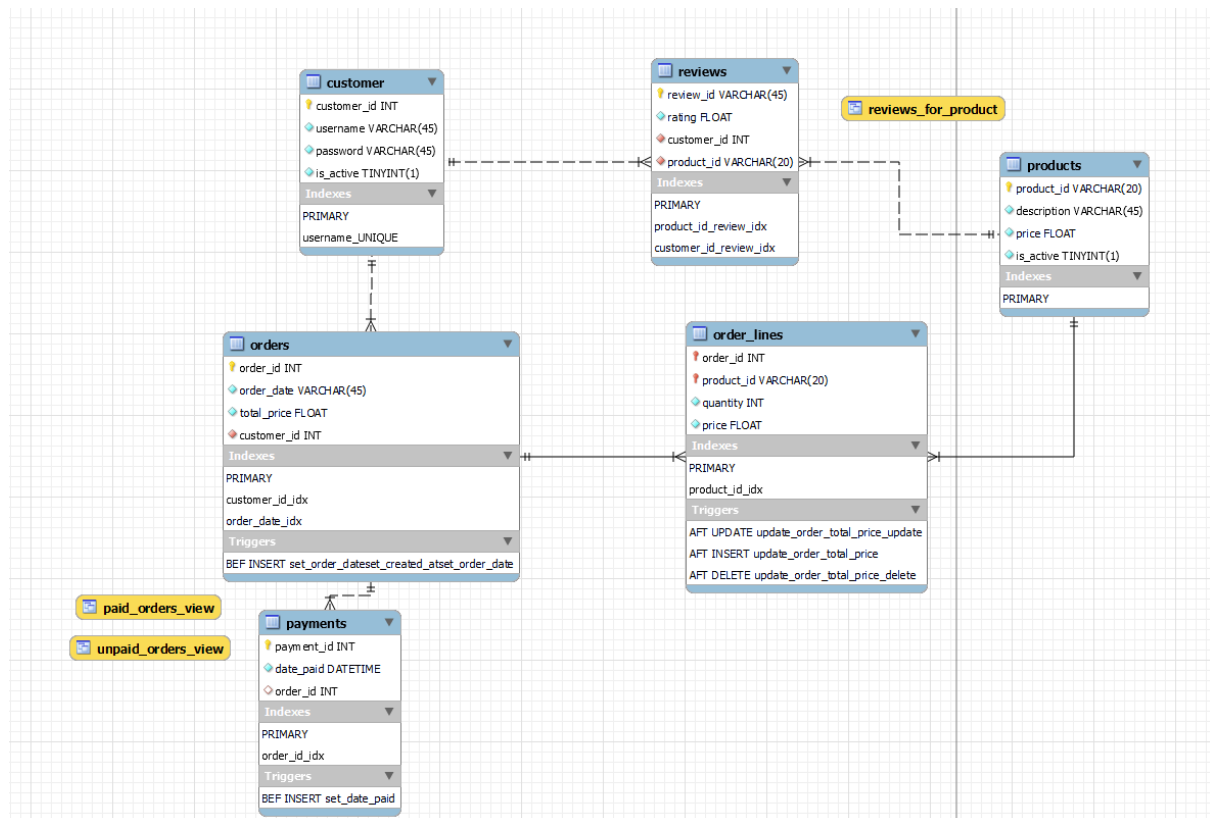
DELIMITER ;
```

Bilag 4

```
use webshop_db;
```

```
CREATE OR REPLACE VIEW unpaid_orders_view AS
SELECT o.order_id, o.order_date, o.total_price, o.customer_id
FROM orders o
LEFT JOIN payments p ON o.order_id = p.order_id
WHERE p.payment_id IS NULL;
```

Bilag 5



Bilag 6

```
def fetch_avg_review_length():
    mongodb_conn = connect_mongodb()
    if mongodb_conn is None:
        raise ConnectionError("Failed to connect to MongoDB database.")
    db = mongodb_conn['webshop_db']
    reviews_collection = db['reviews']
    pipeline = [
        {"$match": {"review_text": {"$type": "string"}}},
        {
            "$project": {
                "word_count": {
                    "$size": {
                        "$filter": {
                            "input": {"$split": ["$review_text", " " ] },
                            "as": "word",
                            "cond": {"$ne": ["$$word", "" ] }
                        }
                    }
                }
            }
        },
        {
            "$group": {
                "_id": None,
                "average_word_count": {"$avg": "$word_count" }
            }
        }
    ]
    result = list(reviews_collection.aggregate(pipeline))
    return result[0]['average_word_count'] if result else 0
```